

CSCE 658 RANDOMIZED ALGORITHMS · MODULE 10 · LECTURE NOTES

Dimension and Sampling

Projecting, hashing, and sampling in high dimensions.

MODULE OUTCOMES

State the Johnson–Lindenstrauss lemma and its consequences.

Explain locality-sensitive hashing.

Explain importance sampling and its variance behaviour.

Explain sampling-based matrix approximation.

Choose a technique for a high-dimensional problem.

1 Johnson–Lindenstrauss

JOHNSON–LINDENSTRAUSS: PROJECT TO $O(\log n / \epsilon^2)$ DIMENSIONS AND KEEP ALL PAIRWISE DISTANCES

For any set of n points in any dimension whatsoever, a random linear map into $O(\log n / \epsilon^2)$ dimensions preserves all pairwise distances to within a factor of $(1 \pm \epsilon)$, with high probability.

Note carefully what the target dimension does *not* depend on: the original dimension. A million-dimensional dataset of a thousand points projects to the same number of dimensions as a thousand-dimensional dataset of a thousand points — which is the remarkable part, and is what makes the result so widely applied.

The proof is Module 02's Chernoff bound plus Module 02's union bound: each individual distance is preserved with probability $1 - 1/n^3$, and there are fewer than n^2 pairs, so the union bound gives $1 - 1/n$ overall. Exactly the per-event-plus-union pattern of Module 03 §1, with the per-event target again set at a power of n .

And the projection is simply a random Gaussian matrix — no structure, no learning, no data dependence whatsoever, which means you can generate it before seeing the data, apply it in a streaming setting, and share it across machines. The data-independence is what makes it usable infrastructure rather than a preprocessing step.

WHAT JL DOES AND DOES NOT GIVE

It preserves *pairwise distances among a specified finite set of points*. Not the geometry of the whole space, and not distances to points added later — though a single fresh point is preserved with high probability, which is enough for most query workloads.

The ϵ^2 in the denominator is harsh: 10% distortion requires about a hundred times more dimensions than 100% distortion, so high accuracy is expensive and the method is best suited to applications tolerating substantial distortion.

And it does not preserve inner products as tightly as distances (the relative error on a small inner product can be large), nor does it help with anything depending on the ambient dimension rather than on distances — volumes, densities, and angles between subspaces are not covered.

But within its scope it is unimprovable: the $O(\log n / \epsilon^2)$ bound is tight (Larsen and Nelson), so no cleverer projection exists — which is a useful thing to know before attempting to improve on it.

2 Locality-sensitive hashing

```
A family H is LOCALITY-SENSITIVE if
  near points collide with probability >= p1
  far points collide with probability <= p2
  with p1 > p2.
```

```
That gap is all you need. Then amplify it:
  AND over k independent hashes -> p1^k vs p2^k
    (widens the ratio; lowers both probabilities)
  OR over L such bands -> 1-(1-p1^k)^L vs the same for p2
    (raises both; steepens the transition)
```

Tuning k and L produces a sharp step at whatever distance you care about -- which is Module 02 section 4's amplification, applied to a similarity threshold rather than to a failure probability.

THE CONSTRUCTIONS

```
HAMMING / JACCARD   MinHash: hash each set by the minimum
                     element under a random permutation.
                     P(collision) = EXACTLY the Jaccard similarity.
COSINE               random hyperplane: take the sign of
                     the dot product with a random Gaussian vector.
                     P(collision) = 1 - theta/pi.
<-- this is exactly CSCE 669 Module 10's
                     Goemans-Williamson rounding, used as a hash.
EUCLIDEAN            project onto a random line and bucket
                     by fixed-width intervals.
```

RESULT: sublinear-time approximate nearest neighbour search, with a stated recall.

The random-hyperplane construction is MAX-CUT's rounding reused as a hash function — the probability that a random hyperplane separates two unit vectors is θ/π , which Goemans and Williamson used to bound a cut and which LSH uses to bound a collision. The same geometric fact, in two different subjects, and noticing it is the kind of connection that makes the material cohere.

3 Importance sampling and matrix approximation

IMPORTANCE SAMPLING REWEIGHTS, AND THE VARIANCE IS EVERYTHING

To estimate an expectation under p , sample from a different distribution q and weight each sample by $p(x)/q(x)$. The estimate is unbiased for any q whose support covers p 's — which is a strong and useful fact.

And the variance depends entirely on how well q matches the shape of $p \cdot f$ — a well-chosen q gives enormous variance reduction and a badly chosen one gives enormous variance *increase*, which is the asymmetry to respect.

The failure mode is a small number of samples carrying huge weights: the estimate becomes dominated by one sample, and the *apparent* variance collapses while the true error does not — because the remaining samples all have negligible weight and therefore agree with each other. **Which is the same silent failure as Module 09 §4's unmixed chain.**

So monitor the effective sample size computed from the weights — the same diagnostic as MCMC uses — and note that CSCE 647 Module 03's importance sampling for light transport and CSCE 642 Module 05 §3's off-policy correction are both exactly this technique, with exactly this failure mode.

SAMPLING FOR MATRIX COMPUTATIONS

Approximate a matrix product by sampling rows of one factor and columns of the other in proportion to their norms — and the error depends on the number of samples taken rather than on the size of the matrices, which is the same scale-independence as JL's.

Which gives randomised low-rank approximation: apply a random projection to reduce the matrix to a small one, decompose the small result exactly, and lift back — with a bound on the error relative to the best possible rank- k approximation.

And it is far faster than a full singular value decomposition, which is why randomised numerical linear algebra is the standard approach for very large matrices (CSCE 676's subject, and CSCE 669's large-scale methods).

With the same structural caveat as always: the error bound holds in expectation or with high probability, and the norms you need to sample by must themselves be cheaply computable — which for a matrix arriving as a stream they may not be, and uniform sampling in place of norm-proportional sampling gives a much weaker bound.

4 Choosing

Problem	Technique	Guarantee
Reduce dimension while keeping distances	Random projection (JL)	$(1 \pm \epsilon)$ on all pairwise distances (§1).

Problem	Technique	Guarantee
Approximate nearest neighbour search	LSH	Sublinear query time with a stated recall (§2).
Estimate an expectation you cannot integrate	Importance sampling	Unbiased; the variance depends on the proposal (§3).
Approximate a large matrix or its decomposition	Randomised SVD	Near-optimal rank-k error, in expectation.
Deduplicate or cluster by set similarity	MinHash	The collision probability is exactly the Jaccard similarity.
Count distinct items	HyperLogLog	Module 05 §1's 2% from 1.5 KB.

MinHash is the one with the cleanest guarantee: the collision probability is *exactly* the Jaccard similarity, with no approximation at all in the expectation — so a MinHash signature of k permutations gives an unbiased estimate whose variance is governed by Module 02's bounds on a sum of k independent indicators. **Which makes it the easiest technique in this module to reason about and the one to reach for when set similarity is the question.**

WHY RANDOMNESS IS UNAVOIDABLE IN HIGH DIMENSIONS

In high dimensions almost every pair of random vectors is almost orthogonal, and almost every pair of points is almost equidistant — the concentration of measure, which is **Module 02's phenomenon applied to geometry** rather than to sums.

Which makes deterministic structure useless: space-partitioning indexes — kd-trees, R-trees, and their relatives — degrade to a linear scan above roughly twenty dimensions, and this is provable rather than an implementation limitation (CSCE 620 Module 07's dimension dependence).

And it makes randomness effective, for the very same reason: a random projection works precisely *because* high-dimensional vectors concentrate — the projection's length distortion is a sum of many small independent contributions, so it concentrates tightly around its mean.

So the curse of dimensionality and the usefulness of randomness are the same fact. Which is the most satisfying observation in this module, and it explains why high-dimensional methods are randomised essentially without exception — not as a convenience, but because concentration is the only structure high dimensions reliably provide.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Blum, Hopcroft & Kannan — Foundations of Data Science (free PDF)	Free in full, and the best treatment of this module — JL, concentration of measure, LSH, and randomised linear algebra in one place.
Andoni & Indyk — Near-Optimal Hashing Algorithms for Approximate Nearest Neighbor (free)	§2's constructions and the amplification analysis.
Halko, Martinsson & Tropp — Finding Structure with Randomness (free)	§3's randomised matrix methods , with the error bounds and practical algorithms. The standard reference.
Leskovec, Rajaraman & Ullman — Mining of Massive Datasets (free PDF)	§2's MinHash and LSH from the practitioner's side , free in full — and the chapter is the clearest introduction available.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Implement a JL random projection** and measure the distance distortion against the predicted $(1 \pm \epsilon)$.
- 2 **Confirm the target dimension does not depend on the original** by projecting from 1000 and from 100,000 dimensions.
- 3 **Plot the required dimension against ϵ** and confirm the ϵ^{-2} scaling.
- 4 **Implement MinHash** and verify the collision probability equals the Jaccard similarity.
- 5 **Implement random-hyperplane LSH** and relate it to CSCE 669's MAX-CUT rounding.
- 6 **Tune k and L** and plot the collision probability against distance. Confirm the step sharpens.
- 7 **Implement importance sampling** with a good and a bad proposal, and compare the variances.
- 8 **Monitor the effective sample size** for the bad proposal and confirm it collapses.
- 9 **Implement randomised SVD** and compare against an exact one for accuracy and time.
- 10 **Measure a kd-tree's query time against dimension** and locate where it degrades to linear.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 State Johnson–Lindenstrauss and say what the target dimension does not depend on.
- 2 What is the proof, in terms of Module 02?
- 3 Name three things JL does not give.
- 4 Define locality-sensitive hashing and explain the amplification.

- 5 Give three LSH constructions and which other course one comes from.
- 6 Explain importance sampling and its failure mode.
- 7 What diagnostic catches it, and where else does it appear?
- 8 How does sampling approximate a matrix, and what is the caveat?
- 9 Give the six-row toolkit table.
- 10 Why are the curse of dimensionality and the usefulness of randomness the same fact?